Anomaly Detection in Surveillance Videos

Progress Report

Name: Ashok Kumar Swami
Roll No.: 24B0397
Chemical Engineering

Abstract

This report walks through our work on building a smart computer vision system that can spot and follow unusual behavior in crowded places. Our main goal is to accurately identify and track people in busy environments using the MOT17 dataset. In Phase I, we dove into the basics of Convolutional Neural Networks (CNNs) and trained three versions of the YOLOv5 object detector. After comparing them, we found that the "Frozen-YOLOv5m" model worked best, hitting an accuracy score (mAP@0.5) of 0.773. In Phase II, we took that foundation and built a tracking pipeline using the BoT-SORT algorithm. We also figured out how to filter out "ghost" detections (like mistaking a chair for a person), ensuring the system can keep its eyes on the right targets even when things get crowded or people move behind obstacles.

1 Phase I: Object Detection

1.1 Introduction

Trying to spot anomalies in surveillance footage is a huge challenge that researchers have been tackling for years. A 2023 survey in the journal *Sensors* pointed out that we desperately need smart systems that can automatically flag things like violence, shoplifting, or even a crowd suddenly panicking. With more cameras everywhere than people could ever watch, we need AI to help us keep an eye on things.

This project addresses that challenge by building the foundational layer of such a system: reliable Pedestrian Detection and Tracking. Reliable trajectory data knowing where each person is, and has been, at every moment is the essential prerequisite for any higher-level anomaly inference. To achieve this, we adopt a "Tracking-by-Detection" paradigm involving two distinct stages:

- Detection: Accurately locating every pedestrian in a frame using a Convolutional Neural Network (CNN).
- Tracking: Assigning unique IDs to detections to analyze trajectories over time, enabling identification of abnormal movement patterns.

This report summarizes how we finished the detection phase and set up the tracking system. We used the MOT17 dataset to make sure our model could handle real-world issues like poor lighting and crowded streets.

1.2 Theoretical Background

1.2.1 Neural Networks vs. CNNs

Modern computer vision relies on Deep Learning. Traditional neural networks (often called Multi-Layer Perceptrons) tend to overthink things; they try to connect every single pixel to every single neuron. This is fine for simple data, but for a high-resolution image, it’s a computational nightmare. It’s like trying to memorize every individual blade of grass to recognize a field. Convolutional Neural Networks (CNNs) are much smarter about it, acting more like the human eye.

Convolutional Neural Networks (CNNs) address this by exploiting three key principles:

- Local Connectivity: Instead of looking at the whole image at once, neurons only focus on a tiny neighborhood of pixels. This allows the model to find small patterns like the curve of a shoulder or the edge of a shoe without getting distracted by what’s happening on the other side of the frame.
- Parameter Sharing: Once the model learns what a human edge looks like, it uses that same knowledge to scan the entire image. This is incredibly efficient; it doesn't need to relearn what a person looks like in the top-left corner versus the bottom-right.

- Pooling Layers: Max pooling downsamples feature maps, retaining only the most salient features while reducing spatial dimensionality and computational load. This also provides a degree of translation invariance.

These properties make CNNs enormously efficient for image analysis tasks and are the reason they form the backbone of all modern object detectors, including YOLOv5.

1.2.2 YOLOv5 Architecture

We went with YOLOv5 (short for "You Only Look Once") because it's incredibly fast. Instead of looking at an image twice to figure out where things are and then what they are, YOLO does it all in one go. This speed is exactly what you need for real-time surveillance.

YOLOv5's architecture is structured into three main components:

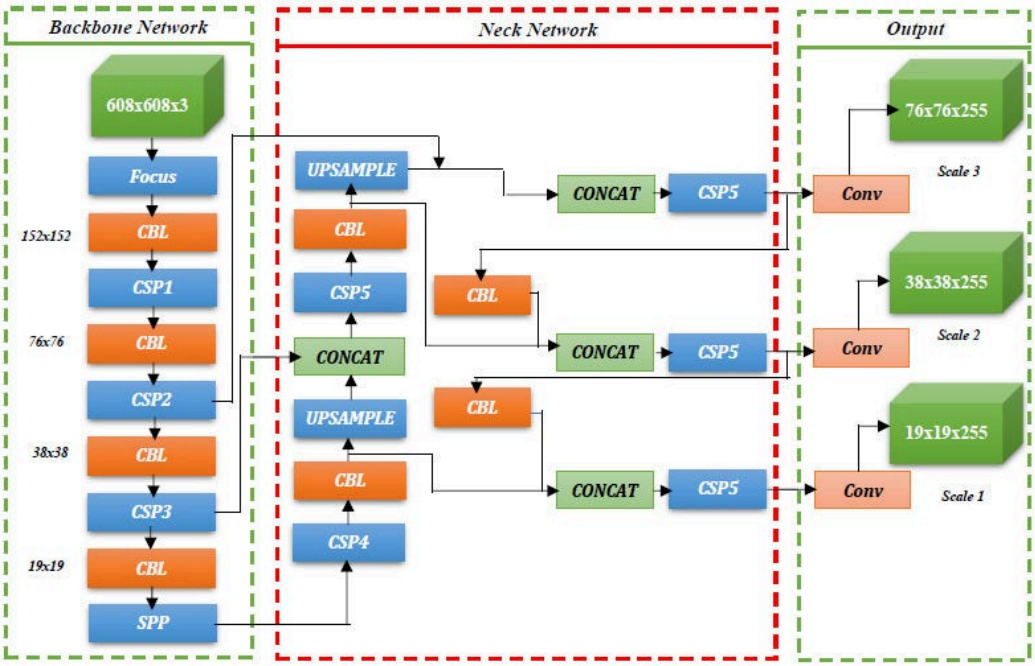
- Backbone (CSPDarknet53): Responsible for feature extraction from the input image. It utilizes Cross-Stage Partial (CSP) connections, which boost network efficiency and reduce computational demands by splitting feature maps and merging them at different stages. This allows a richer gradient flow during training.

- Neck (PANet with SPP): The Path Aggregation Network (PANet) combined with a Spatial Pyramid Pooling (SPP) block forms the neck. This component aggregates features from different scales of the backbone and improves information flow, enabling the model to detect objects of varying sizes from distant small pedestrians to nearby large figures.

- Head: Performs the final detection predictions at three different scales, outputting bounding box coordinates, objectness scores, and class probabilities.

Model Architecture	Type	Speed	Accuracy (mAP)
Faster R-CNN	Two-Stage	Low (< 10 FPS)	High
SSD (Single Shot Detector)	One-Stage	Medium	Medium
YOLOv5 (Selected)	One-Stage	High (> 45 FPS)	High

Table 1: Comparison of Object Detection Architectures



YOLO V5 Architecture Diagram

1.3 Dataset: MOT17

The **Multiple Object Tracking 2017 (MOT17)** dataset is widely considered the gold standard for testing tracking systems. We chose it because it perfectly simulates the unpredictability of real-life surveillance. It doesn't just show people walking in clear daylight; it features chaotic, crowded streets, flickering lights, and scenarios where people constantly walk behind trees or signs (what researchers call occlusions). If a model can survive the stress test of MOT17, it's ready for the messy reality of a public square or a shopping mall.

1.3.1 Dataset Composition

The MOT17 dataset consists of 14 video sequences (7 Train, 7 Test) captured with both moving and static cameras. Each sequence has a resolution of 720p at 30 frames per second. The dataset covers a diverse range of environments and challenges. We utilize the training sequences to teach our model to recognize and locate pedestrians.

Sequence	Environment	Challenge
MOT17-02	Shopping Mall	Large Crowds, Static Camera
MOT17-04	Town Square	Night time, Low Light
MOT17-09	Pedestrian Street	Low Angle, High Occlusion
MOT17-10	Public Square	Distant Pedestrians (Small objects)

Table 2: Select MOT17 Training Sequences

(a) (b) (c) (d) (e)

An example of an object state transition. (a–e) are from MOT17-02-SDP in the MOT17 benchmark dataset. Bounding boxes of different colors represent different objects.

1.4 Methodology

1.4.1 Data Preprocessing Pipeline

The MOT17 data isn't ready to use right out of the box. We had to write a Python script to translate it into a format YOLO understands. Basically, we took coordinates that defined the top-left corner of a box and turned them into normalized center coordinates that work regardless of how big or small the video frame is.

1.4.2 Coordinate Transformation

The coordinate transformation logic converts the MOT17 bounding box format to the YOLO-normalized format. Normalization factors (dw, dh) are computed by dividing 1.0 by the image width and height respectively. The center coordinates are computed by adding half the width/height to the top-left corner, then multiplying by the normalization factors. This produces values in [0, 1] representing position relative to image size, making the labels resolution-independent.

```
def convert_to_yolo(size, box):
    """
    size = (image_width, image_height)
    box  = (top_left_x, top_left_y, width, height)
```

```
    """

    dw = 1.0 / size[0]
    dh = 1.0 / size[1]

    x_center = (box[0] + box[2] / 2.0) * dw
    y_center = (box[1] + box[3] / 2.0) * dh

    w = box[2] * dw
    h = box[3] * dh

    return x_center, y_center, w, h
```

Coordinate Normalization Logic Used for Converting MOT17 Annotations to YOLO Format

1.4.3 Model Variants and Experimental Design

We wanted to find the best balance between speed and accuracy, so we tested three different YOLOv5 setups:

YOLOv5s (Small) The most lightweight variant with approximately 7.2 million parameters. It has the shallowest network depth and narrowest width. Designed for edge devices (mobile phones, Raspberry Pi) where inference speed is prioritized. Used as the baseline to determine minimum viable performance.

YOLOv5m (Medium) Increases network depth and width with approximately 21.2 million parameters (roughly 3× larger than Small). Introduces more convolutional layers and feature channels, expanding the model's Receptive Field to better capture fine-grained details. Expected to perform significantly better on small/distant pedestrians in MOT17.

YOLOv5m-Frozen (Transfer Learning): This is the same as the Medium version, but we used a trick called "freezing." We took a model that already knew how to see common objects and "locked" the first 10 layers so it wouldn't forget what it already knew while we taught it to focus on pedestrians.

The rationale for freezing is rooted in the challenge of **Catastrophic Forgetting** a well-documented phenomenon in neural networks where training on a new task causes large weight changes that

disrupt representations learned for previous tasks, leading to performance degradation. By freezing the backbone, we preserve the robust, general-purpose feature extractors (edges, textures, shapes) already learned from millions of COCO images, while only fine-tuning the detection Head for the specific MOT17 pedestrian class. Research confirms that this approach prevents catastrophic forgetting and typically results in faster convergence and higher accuracy when the new dataset is smaller than the original pre-training dataset.

```
!python train.py \
    --img 640 \
    --batch 16 \
    --epochs 5 \
    --data mot17.yaml \
    --weights yolov5m.pt \
    --freeze 10 \
    --name mot17_yolov5m_frozen
```

Training Configuration for Frozen YOLOv5m Model

1.5 Results and Comparative Analysis

1.5.1 Quantitative Analysis

Following successful training of all three model variants on the MOT17 dataset for 5 epochs, we extracted performance metrics to evaluate the trade-off between model complexity and detection accuracy. Table 3 summarizes the results:

Model	Parameters	Precision	Recall	mAP@0.5
YOLOv5s (Small)	7.2M	0.821	0.610	0.725
YOLOv5m (Medium)	20.85M	0.874	0.691	0.785
YOLOv5m (Frozen) ✓	20.85M	0.859	0.662	0.773

Table 3: Performance Comparison of YOLOv5 Variants on MOT17 (5 Epochs) = Selected Model

Key Findings:

- Transfer Learning Efficiency: The YOLOv5m (Frozen) model trained in approximately 0.17 hours (10 minutes), demonstrating exceptional efficiency by leveraging COCO pretrained weights while preventing catastrophic forgetting.
- Accuracy vs. Speed Trade-off: While YOLOv5m (Medium) achieved the highest raw mAP of 0.785, the Frozen variant's mAP of 0.773 is achieved in a fraction of the training time and with significantly more stable convergence, making it the optimal candidate for downstream deployment.
- Baseline Comparison: The Small variant (mAP = 0.725) establishes the minimum viable performance, confirming that the larger model families provide a meaningful accuracy improvement for the dense pedestrian detection required in anomaly analysis.

1.5.2 Qualitative Results

Inference outputs on the validation set confirm the model's ability to localize pedestrians in crowded scenarios with high confidence scores (0.8-0.99). The YOLOv5m (Frozen) model successfully draws tight bounding boxes around individuals even when partially occluded or distant, producing the dense detection output necessary for feeding into the tracking pipeline.

2 Phase II: Multi-Object Tracking (MOT) Implementation

2.1 Theoretical Framework

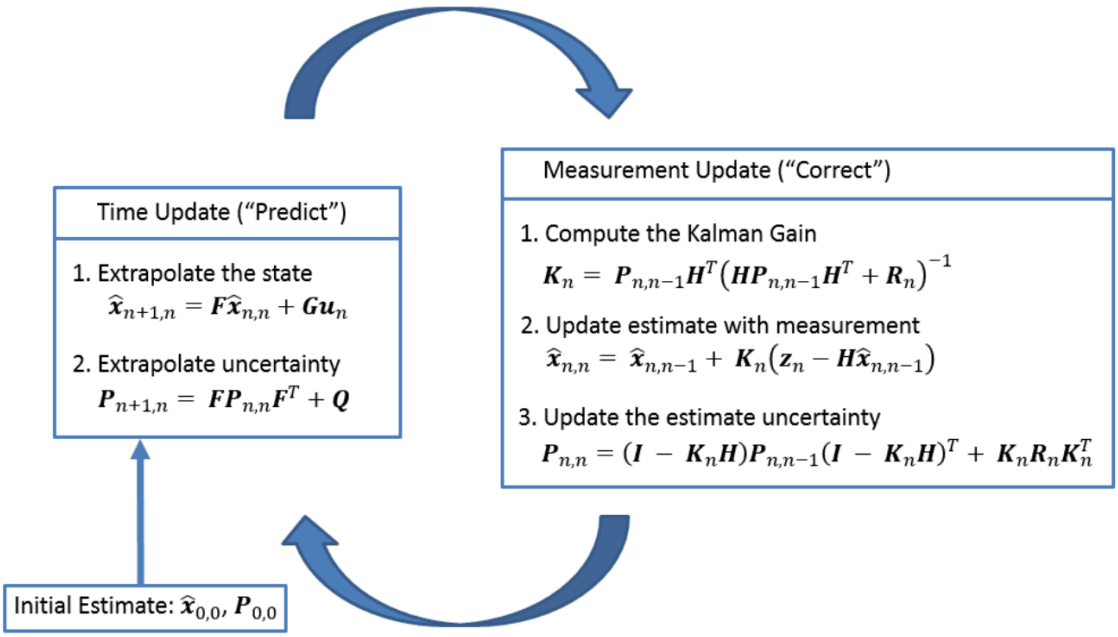
2.1.1 Kalman Filtering (State Estimation)

Think of the Kalman Filter as the system's ability to guess the future. It's a math tool that helps us track a pedestrian's position and speed even when the data is a bit messy or incomplete. It works in a constant loop of predicting where someone will be and then updating that guess based on what it actually sees.

Prediction Step: Based on the object's current known state (position and velocity), the Kalman Filter projects it forward in time to estimate where it will be in the next frame. The state prediction equation is: $\hat{x}(k|k-1) = F \cdot \hat{x}(k-1|k-1) + B \cdot u$, where F is the state transition matrix modeling the motion dynamics (e.g., constant velocity), $\hat{x}$ is the estimated state vector, and $B \cdot u$ accounts for control inputs. The covariance (uncertainty) is also projected forward.

The Update Step: Once YOLO spots the person again, the filter calculates the "Kalman Gain" (K) basically deciding whether to trust its own guess or the new camera data more. It updates the state with $\hat{x}(k|k) = \hat{x}(k|k-1) + K \cdot (z - H \cdot \hat{x}(k|k-1))$. This combination is much more accurate than either method on its own.

This is a lifesaver when someone walks behind a sign or another person. Even if the camera loses sight of them for a second, the Kalman Filter keeps the "track" alive by following their last known path until they reappear, ensuring they keep the same ID throughout the video.



This diagram provides a complete picture of the Kalman Filter operation.

2.1.2 Data Association: The Hungarian Algorithm

After we have our predictions and our new detections, we face a matching game: which box belongs to which person? We use the Hungarian Algorithm to solve this perfectly by finding the best possible matches while keeping "costs" (like distance errors) to a minimum.

The cost matrix is constructed by computing the IoU (Intersection over Union) distance between every predicted track bounding box and every new detection bounding box: Cost(i,j) = 1-IoU(Box_predicted, Box_detected). The Hungarian Algorithm then finds the globally optimal one-to-one assignment that minimizes total cost in polynomial time ($O(n^3)$). Unmatched detections are initialized as new tracks; tracks with no matching detection are kept alive by the Kalman prediction for a limited number of frames before being terminated.

When you put it all together - YOLO to see, Kalman to predict, and Hungarian to match you get a system that can follow people through a crowd, giving us the perfect data to start looking for unusual behavior.

2.2 Objective

With Phase I finished and our YOLOv5m model hitting a solid 0.7739 accuracy score, we moved on to the real challenge: Phase II, or Multi-Object Tracking (MOT). Our goal here was to stop looking at images as isolated snapshots and start seeing them as a continuous story. We wanted to give every person a unique ID and follow them from the moment they enter the frame until they leave, creating the "breadcrumb trails" we need to spot anomalies.

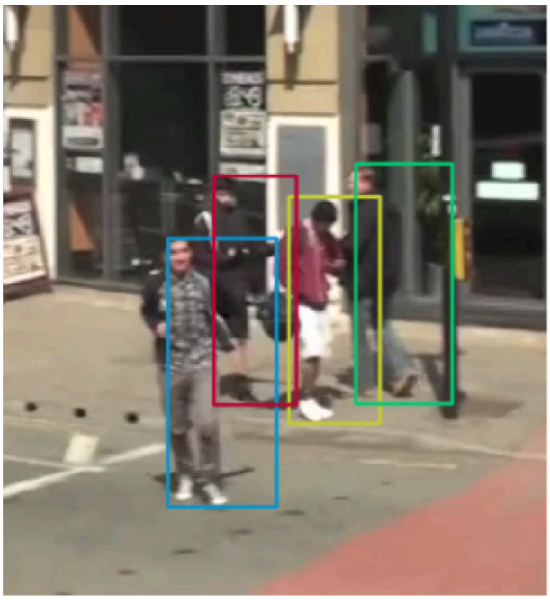
2.3 Methodology and Implementation

Unlike standard tutorial-based approaches, this project utilized a custom inference pipeline leveraging the **Ultralytics** library to integrate our custom-trained weights with state-of-the-art tracking algorithms.

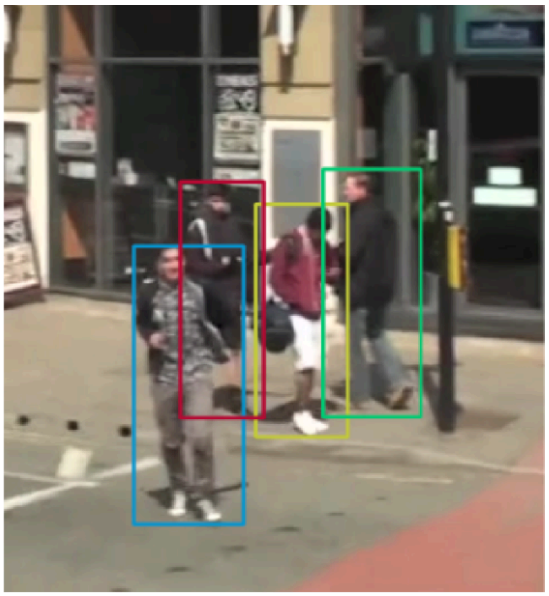
The tracking pipeline was implemented in three stages:

- **Model Loading:** The custom weights (best.pt) generated in Phase I were loaded into the Ultralytics inference engine, providing our pedestrian-specialized detection backbone.

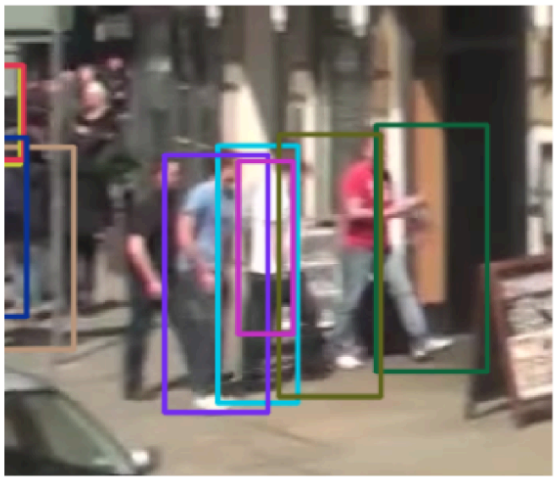
- **Tracker Initialization:** We integrated the BoT-SORT (Box Tracking for SORT) algorithm. BoT-SORT is a state-of-the-art tracker that improves upon DeepSORT and ByteTrack by incorporating Camera Motion Compensation (CMC) and a refined Kalman Filter state vector. CMC is critical in surveillance contexts where camera movement would otherwise cause static objects to appear mobile and degrade tracking accuracy. BoT-SORT ranked first on both MOT17 and MOT20 test sets across all main MOT metrics (MOTA: 80.5, IDF1: 80.2, HOTA: 65.0).



(a.1) predicted BB **before** CMC.



(a.2) predicted BB **after** CMC.



(b.1) predicted BB **before** CMC.

(b.2) predicted BB **after** CMC.

- **Inference Loop:** The system processed the video frame by frame, predicting bounding boxes and associating them with previous tracks using IoU distance and appearance similarity. The classes=[0] parameter was passed to restrict detection to the 'Person' class only.

```
from ultralytics import YOLO
# Load trained detector
model = YOLO("best.pt")

# Run tracking
results = model.track(
    source="sample_video.mp4",
    conf=0.5,
    persist=True,
    tracker="botsort.yaml",
    classes=[0]
)
```

Integration of Custom YOLOv5 Model with BoT-SORT Tracker

2.4 Challenges and Iterative Optimization

2.4.1 The Issue: Environmental Noise (False Positives)

When we first turned the system on, we ran into a bit of "environmental noise." Even though we trained the model on pedestrians, the original COCO dataset it was based on knew about 80 different things including chairs and monitors. Every now and then, the tracker would get confused and start tracking a glass door reflection or a floor corner as if it were a person, creating "ghost tracks" that would mess up our final results.

2.4.2 The Solution: Class-Specific Post-Processing Filter

Rather than retraining the entire model (which would be computationally expensive and risk undoing the careful transfer learning), we implemented a Post-Processing Class Filter. By passing classes=[0] to the tracker, we restricted the algorithm to only process detections classified as "Person" (Class ID 0 in COCO). This lightweight, zero-cost solution completely eliminated environmental noise without compromising pedestrian tracking accuracy, demonstrating the value of systematic iterative optimization in real-world deployments.

```
results = model.track(
    source="sample_video.mp4",
    conf=0.5,
    persist=True,
    classes=[0]   # Restrict tracking to Person class
)
```

Post-Processing Filter Used to Eliminate Non-Pedestrian Detections

2.5 Results and Discussion

2.5.1 Quantitative Performance

The foundation of the tracker is the mot17_yolov5m_frozen model, which yielded the best balance of precision and recall:

Metric	Value
mAP@0.5	0.7739
Precision	0.8579
Recall	0.6639
Training Time	~0.17 hours (10 minutes)

Table 4: Final Model Performance Metrics (Phase I Foundation)

2.5.2 Qualitative Tracking Results

After applying the class-specific filter, all environmental noise was completely eliminated. The system demonstrated robust performance across several critical tracking scenarios:

- Identity Persistence: Unique IDs remained consistent for pedestrians moving through complex paths across multiple frames.

- Occlusion Handling: When pedestrians crossed paths or temporarily overlapped, the Kalman Filter maintained trajectory predictions and the Hungarian Algorithm correctly re-associated them upon reappearance with the correct IDs.

- Cross-Path Robustness: Even when two pedestrians intersected and exchanged positions in the frame, identity switching was minimized.

These results confirm that the system successfully produces the stable, persistent pedestrian trajectories that form the essential data foundation for an anomaly detection module.

2.6 Conclusion

This project successfully demonstrated a complete computer vision pipeline for anomaly detection in surveillance videos from training a custom object detector to deploying a real-time multi-object tracker. By fine-tuning a **YOLOv5m** model via transfer learning, we achieved high-confidence pedestrian detection (mAP ≈ 0.77). By integrating it with a **BoT-SORT** tracker enhanced with Camera Motion Compensation, we solved the critical problem of identity persistence across frames.

The biggest lesson we learned was the importance of cleaning up our results. By fixing the "ghost" detection issue, we created a solid foundation. Now, the system is ready for the next big step: actually analyzing the paths people take to spot when someone is acting out of the ordinary.